

ENGINEERING DECISIONS. STACK RATIONALE. ARCHITECTURE PRINCIPLES.

Technology Stack Reference

— Every technology chosen for a reason. —

Full-Stack Engineering

Next.js · React · TypeScript · FastAPI · PostgreSQL

Autonomous AI Workflows

LLM orchestration · RAG pipelines · Playwright RPA

Robotic Process Automation

Headless bots · 24/7 execution · legacy systems

Transparent Pricing

Published rates. Free prototype before any payment.

Engineering-First

The founder writes production code on every project.

Deterministic AI

Strict JSON scaffolding on every LLM integration.

8+

Platforms

Omni-channel publishing

100%

Custom-Built

No templates. Ever.

₹8,999

Starting From

Digital Launch — 4 weeks

COMPANY

Logic Intelligence Technologies

FOUNDER

Vikash Saravanan

LOCATION

Coimbatore, Tamil Nadu, India

WEBSITE

logicintelligencetechnologies.in

“Where Logic Meets Innovation” — every system we architect, every line we deploy.

Visit logicintelligencetechnologies.in to explore services, case studies, and pricing packages.

CORE SERVICES

Full-Stack Web Apps

AI & LLM Integration

Robotic Process Automation

PostgreSQL & Supabase

CI/CD & DevOps

E-Commerce Platforms

RAG Pipelines

API Engineering

SECTION 01

Frontend Engineering Stack

WHY NEXT.JS?

Next.js 15 with React 19 gives us the most powerful frontend architecture available: SSR for SEO and dynamic data, SSG for sub-100ms static performance, App Router for nested layouts and streaming, and React Server Components for zero client-bundle server fetching. We choose Next.js not for trend — but because it is the correct engineering choice for every use case we solve.

FRONTEND TECHNOLOGY DECISIONS

TypeScript (Strict Mode)

Type safety catches bugs at compile time. Reduces runtime errors by 70%+ in production.

Tailwind CSS

Utility-first CSS eliminates specificity wars. Dark mode with class strategy. Consistent spacing scale.

shadcn/ui + Radix UI

Accessible, unstyled primitives. Zero runtime CSS overhead. ADA compliance built in.

Framer Motion

Production-grade animation library. Respects prefers-reduced-motion. Hardware-accelerated transforms.

React Hook Form + Zod

Type-safe form handling with runtime schema validation. Same Zod schema shared with backend.

SECTION 02

Backend Engineering Stack

WHY FASTAPI?

FastAPI generates OpenAPI docs automatically, enforces Pydantic type validation on every request and response, and handles 10,000+ concurrent requests with async Python. Combined with Celery for distributed task queues and Redis for caching, it powers our AI orchestration layers and client API backends.

BACKEND TECHNOLOGY DECISIONS

Pydantic v2

Rust-backed validation. Schema shared between API and AI output validation layers.

Celery + Redis

Distributed task queue for RPA bots and AI pipelines. Retry logic built in.

Docker + Docker Compose

Reproducible environments. One-command local dev setup. Cloud-identical staging.

Pytest + Coverage

Test-driven development. 80%+ coverage target. CI blocks merge on failing tests.

SECTION 03

Database Architecture

Supabase PostgreSQL is the foundation of every LIT application. We choose it for its open-source ethos, enterprise-grade features, and the ability to write SQL-level Row Level Security that no application-layer check can override.

DATABASE DESIGN PRINCIPLES

- ◆ **RLS First**
Every table gets an RLS policy before any application code touches it.
- ◆ **Normalisation**
3NF minimum. Denormalise only with explicit justification documented in schema comments.
- ◆ **Index Strategy**
Explain Analyse on every query with > 1000 rows. Partial indexes for soft-delete patterns.
- ◆ **Migration Discipline**
Supabase migrations versioned in Git. Never manual SQL changes in production.
- ◆ **pgvector for AI**
Native vector embeddings for RAG pipelines. Cosine similarity queries at sub-100ms.

SECTION 04

AI & DevOps

AI INTEGRATION PHILOSOPHY

AI is a controlled input/output system inside a deterministic software architecture. Every LLM call uses JSON mode, Pydantic validation, and retry loops. The AI component can be swapped or disabled without breaking the surrounding system. This is the only responsible way to deploy production AI.

DEVOPS STANDARDS

- ◆ **GitHub Actions CI**
Lint Type Check Test Build on every PR. No merge without green.
- ◆ **Environment Variables**
dotenv for local. Vercel env for production. Supabase vault for secrets. Zero hardcoded credentials.
- ◆ **Vercel + AWS Edge**
Vercel for Next.js frontend. AWS EC2 for FastAPI/Docker backend. Global edge CDN.
- ◆ **Monitoring**
Vercel Analytics, Supabase Studio, and custom health endpoints on every production service.